Assignment- 16 .4

Name :B. MANISHWAR

Hall Ticket No. : 2303A51276

Batch No. : 05

Task 1: Database Schema Design for an

Online Course Platform

Scenario

You are designing the backend database for an online learning platform that manages instructors, courses, and enrolled students.

Task Description

• Use AI tools to generate SQL CREATE TABLE statements for the following entities:

o Instructors

o Courses o

Students

• **Include: o** Primary keys o Foreign key relationships o

Appropriate data types

• Ensure the schema follows basic normalization principles.

Expected Output

• All tables are created successfully

• Foreign key constraints enforce valid relationships

• Schema is logically structured and normalized

Prompt:

Generate SQL CREATE TABLE statements for an online course platform.
Tables: Instructors, Courses, Students.
Include primary keys, foreign keys, and proper data types. Ensure normalization.

AI Generated Code:

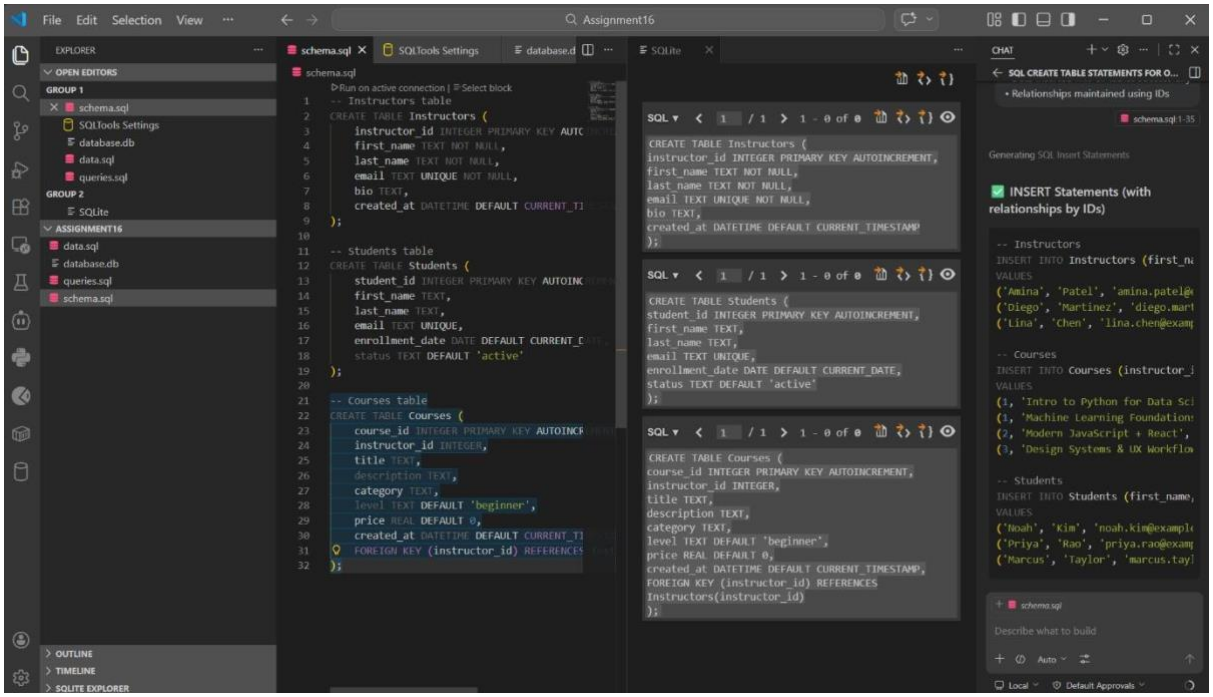
```
-- Instructors table
```

```
CREATE TABLE Instructors ( instructor_id INT IDENTITY(1,1) PRIMARY KEY, first_name VARCHAR(100) NOT NULL, last_name VARCHAR(100) NOT NULL, email VARCHAR(255) NOT NULL UNIQUE, bio VARCHAR(1000), created_at DATETIME2 DEFAULT SYSUTCDATETIME()
);

-- Students table
CREATE TABLE Students ( student_id INT IDENTITY(1,1) PRIMARY KEY, first_name VARCHAR(100) NOT NULL, last_name VARCHAR(100) NOT NULL, email VARCHAR(255) NOT NULL UNIQUE, enrollment_date DATE NOT NULL DEFAULT CAST(GETDATE() AS DATE), status VARCHAR(30) NOT NULL DEFAULT 'active'
);

-- Courses table
CREATE TABLE Courses ( course_id INT IDENTITY(1,1) PRIMARY KEY, instructor_id INT NOT NULL, title VARCHAR(255) NOT NULL, description VARCHAR(2000), category VARCHAR(100), level VARCHAR(50) NOT NULL DEFAULT 'beginner', price DECIMAL(10,2) NOT NULL DEFAULT 0.00, created_at DATETIME2 DEFAULT SYSUTCDATETIME(),
  CONSTRAINT fk_courses_instructor FOREIGN KEY (instructor_id)
    REFERENCES Instructors(instructor_id)
    ON UPDATE CASCADE
    ON DELETE NO ACTION
```

);



Explanation:

- Each table has a **Primary Key**
- **Foreign Keys** maintain relationships:
    - Courses → Instructor ○ Enrollments → Students & Courses
- Separate **Enrollments table** avoids redundancy → ensures **Normalization (3NF)**

Task 2: Populate the Database with Sample Records

Scenario

To test the system, the platform needs initial sample data.

Task Description

• Use AI assistance to generate INSERT statements that add:

o At least 3 instructors

o At least 4 courses o At least 3 students

• Verify correctness using SELECT queries.

Expected Output

• Sample data inserted without errors

• SELECT * queries display the inserted records correctly

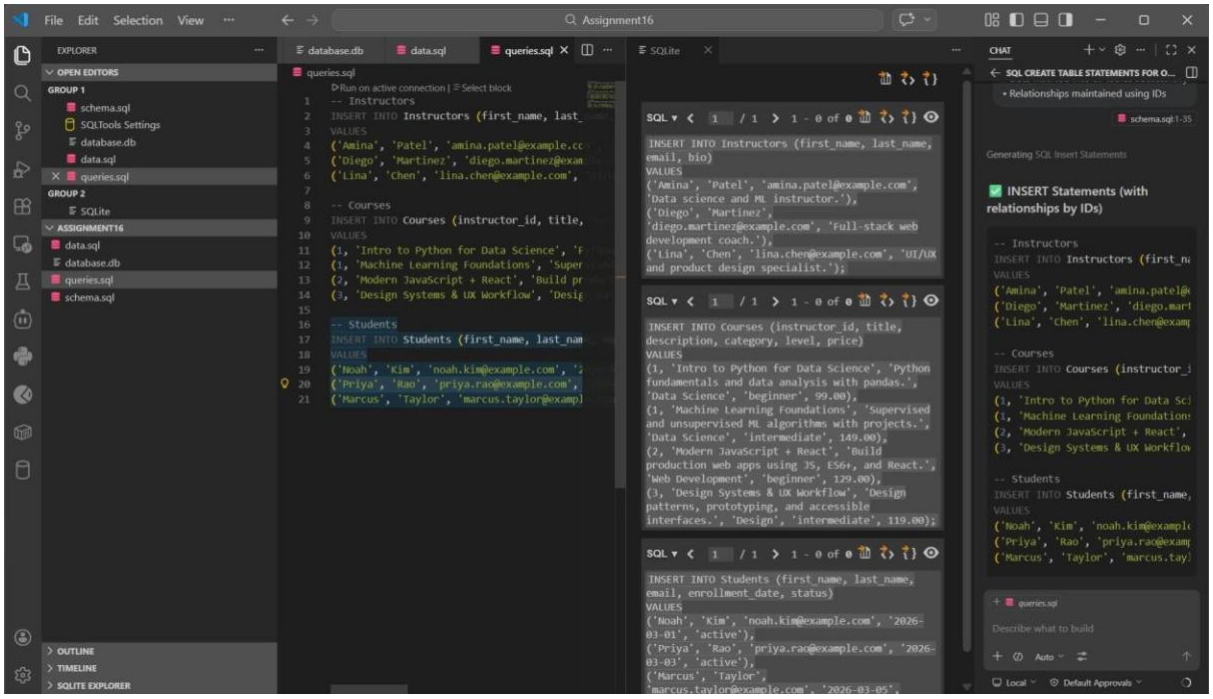
Prompt:

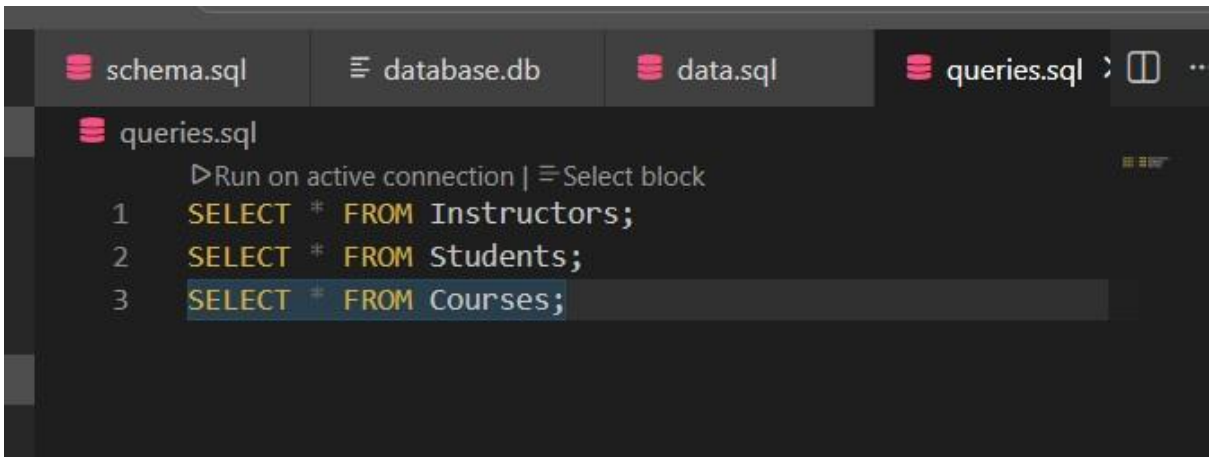
Generate SQL INSERT statements for:

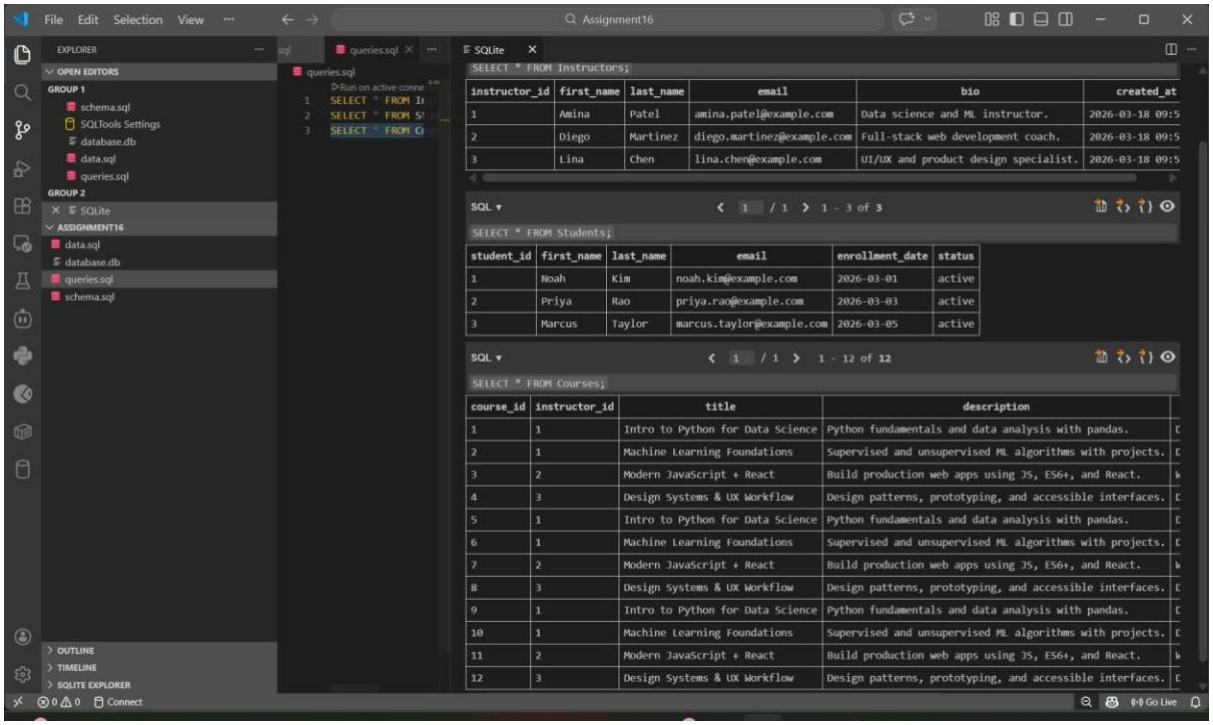
3 instructors, 4 courses, 3 students.

AI Generated Code:

```
-- Instructors
INSERT OR IGNORE INTO Instructors (first_name, last_name, email, bio)
VALUES
('Amina', 'Patel', 'amina.patel@example.com', 'Data science and ML instructor.'),
('Diego', 'Martinez', 'diego.martinez@example.com', 'Full-stack web development coach.'),
('Lina', 'Chen', 'lina.chen@example.com', 'UI/UX and product design specialist.');
 -- Courses
INSERT OR IGNORE INTO Courses (instructor_id, title, description, category, level, price)
VALUES
(1, 'Intro to Python for Data Science', 'Python fundamentals and data analysis with pandas.', 'Data Science', 'beginner', 99.00),
(1, 'Machine Learning Foundations', 'Supervised and unsupervised ML algorithms with projects.', 'Data Science', 'intermediate', 149.00),
(2, 'Modern JavaScript + React', 'Build production web apps using JS, ES6+, and React.', 'Web Development', 'beginner', 129.00),
(3, 'Design Systems & UX Workflow', 'Design patterns, prototyping, and accessible interfaces.', 'Design', 'intermediate', 119.00);
 -- Students
INSERT OR IGNORE INTO Students (first_name, last_name, email, enrollment_date, status)
VALUES
('Noah', 'Kim', 'noah.kim@example.com', '2026-03-01', 'active'),
('Priya', 'Rao', 'priya.rao@example.com', '2026-03-03', 'active'),
('Marcus', 'Taylor', 'marcus.taylor@example.com', '2026-03-05', 'active');
```


File Edit Selection View
Assignment16
EXPLORER
OPEN EDITORS
GROUP 1
schema.sql
SQLTools Settings
database.db
data.sql
queries.sql
GROUP 2
SQLite
ASSIGNMENT16
data.sql
database.db
queries.sql
schema.sql
OUTLINE
TIMELINE
SQLITE EXPLORER
database.db
data.sql
queries.sql
SQLite
CHAT
SQL CREATE TABLE STATEMENTS FOR O...
• Relationships maintained using IDs
schema.sql:1-35
Generating SQL Insert Statements
INSERT Statements (with relationships by IDs)
Describe what to build
Auto
Local
Default Approvals


schema.sql
database.db
data.sql
queries.sql
queries.sql
Run on active connection | Select block
1 SELECT * FROM Instructors;
2 SELECT * FROM Students;
3 SELECT * FROM Courses;



Explanation

- Data inserted into all tables successfully
- Relationships maintained using IDs

Task 3: Implement Core Database

Operations (CRUD)

Scenario

The platform administrator needs to manage course and student information.

Task Description

Using AI-generated SQL, perform the following operations:

• Retrieve all courses offered

• Update a student's email address

• Remove a course from the database

• Verify each operation using appropriate SELECT statements

Expected Output

• Updated data is reflected correctly

• Deleted records are no longer visible

• No unintended data loss occurs **Prompt:**

Generate SQL queries for:

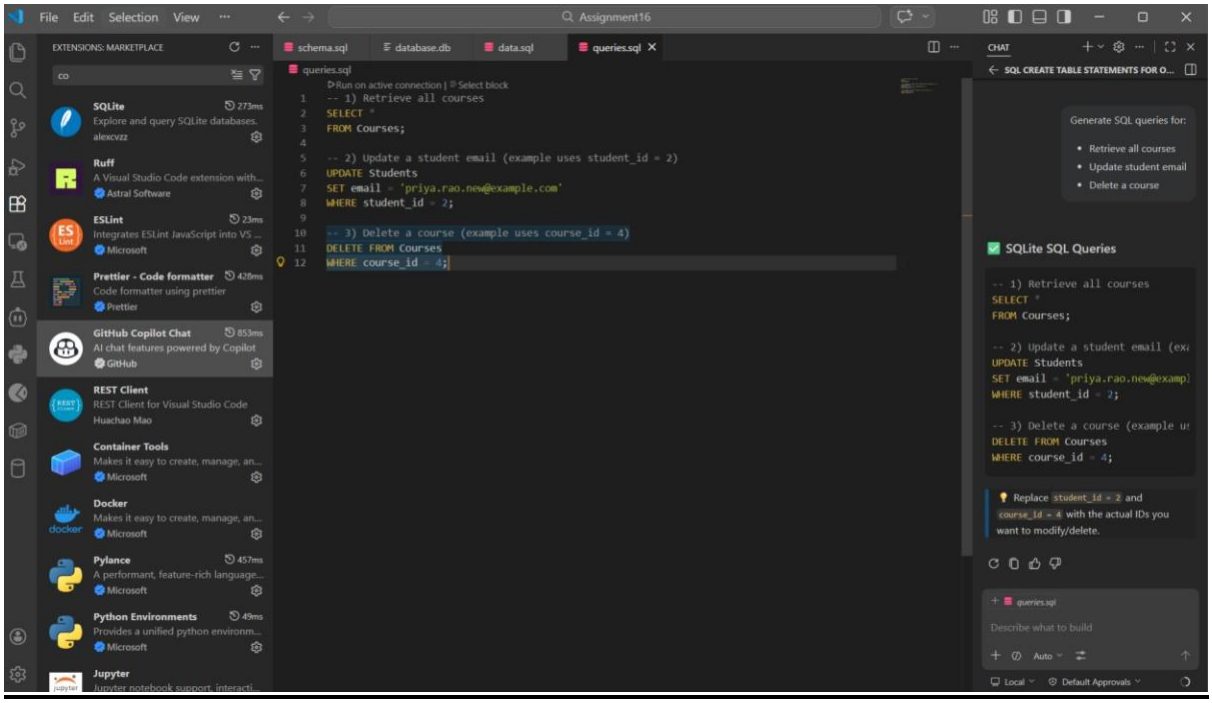
- Retrieve all courses

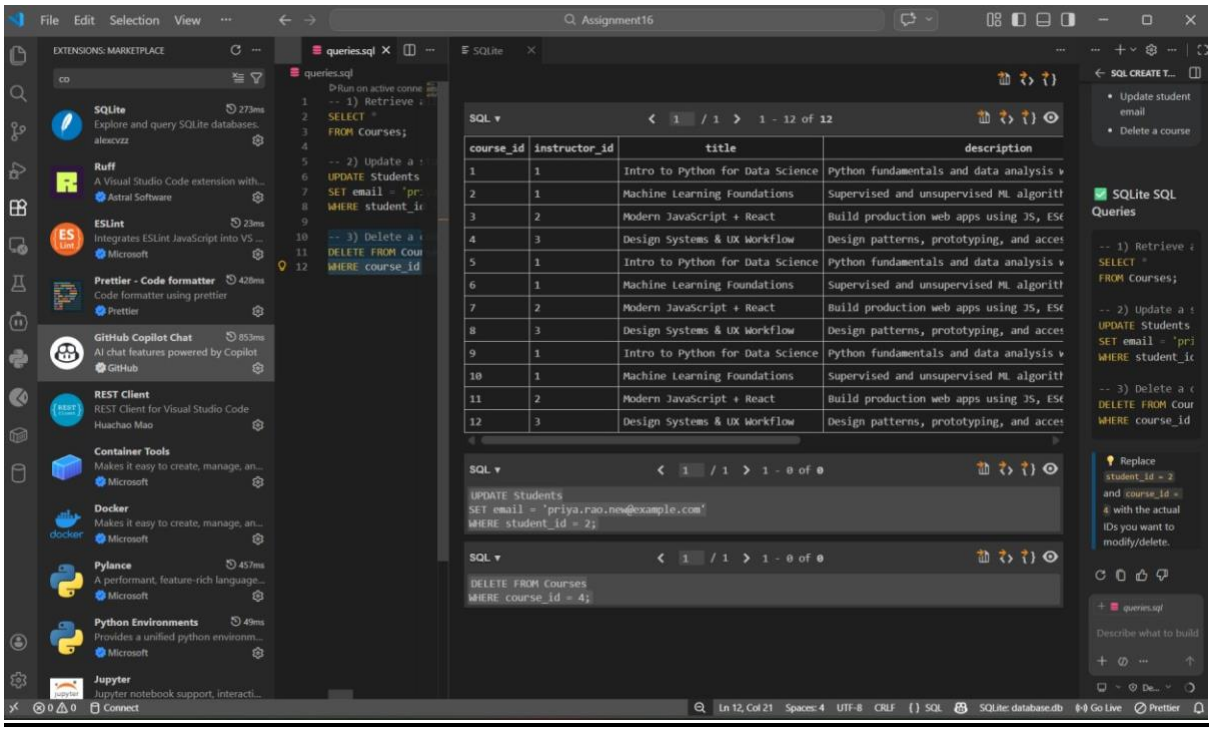
- Update student email

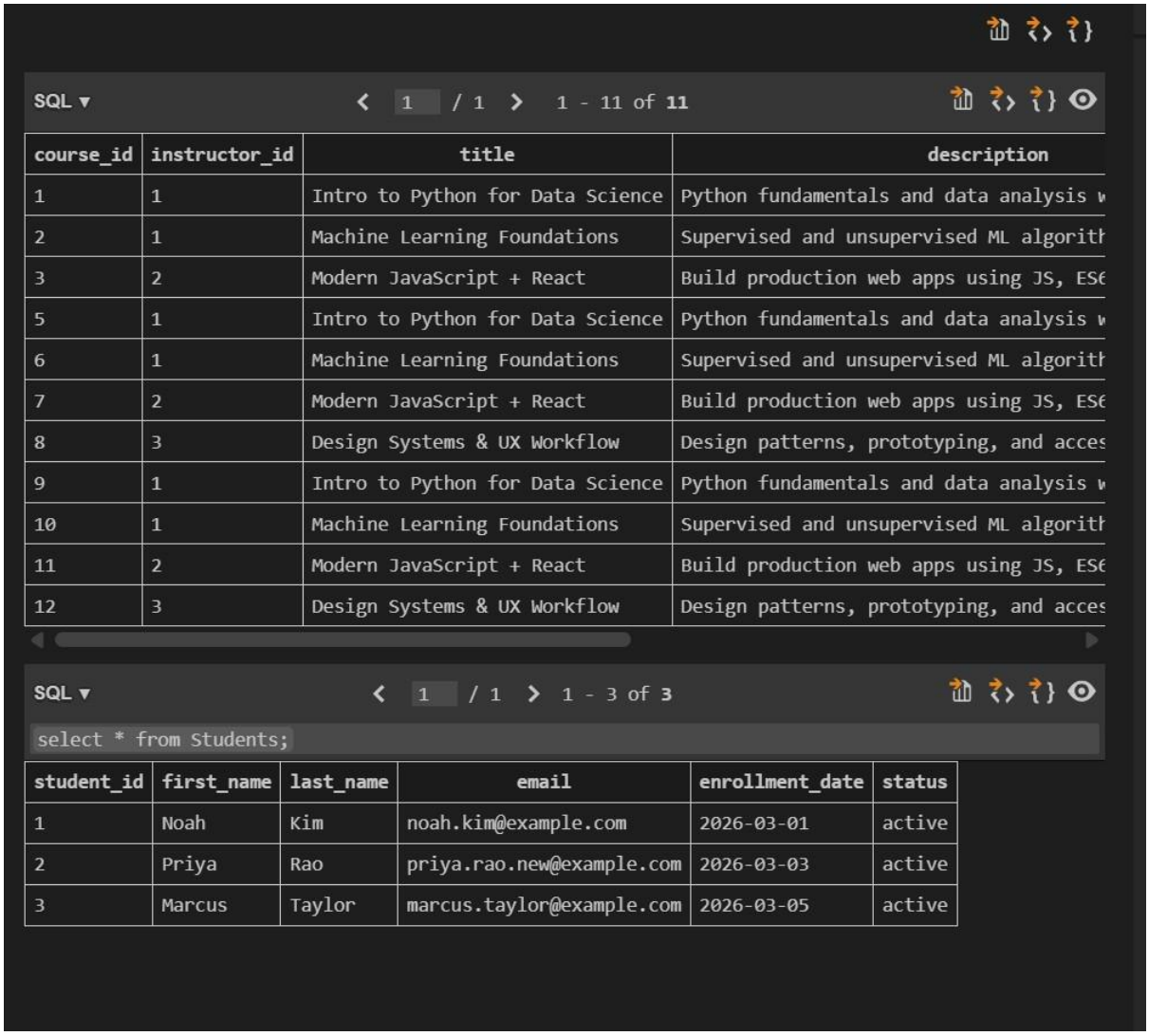
- Delete a course

AI Generated Code:

```
-- 1) Retrieve all courses
SELECT *
FROM Courses;
 -- 2) Update a student email (example uses student_id = 2)
UPDATE Students
SET email = 'priya.rao.new@example.com'
WHERE student_id = 2;
 -- 3) Delete a course (example uses course_id = 4)
DELETE FROM Courses
WHERE course_id = 4;
```


File Edit Selection View
Assignment16
EXTENSIONS: MARKETPLACE
schema.sql
database.db
data.sql
queries.sql
CHAT
SQL CREATE TABLE STATEMENTS FOR O...
Generate SQL queries for:
Retrieve all courses
Update student email
Delete a course
SQLite SQL Queries
-- 1) Retrieve all courses
SELECT *
FROM Courses;
-- 2) Update a student email (example uses student_id = 2)
UPDATE Students
SET email = 'priya.rao.new@example.com'
WHERE student_id = 2;
-- 3) Delete a course (example uses course_id = 4)
DELETE FROM Courses
WHERE course_id = 4;
Replace student_id = 2 and course_id = 4 with the actual IDs you want to modify/delete.
Describe what to build
SQLite
Explore and query SQLite databases.
Ruff
A Visual Studio Code extension with...
ESLint
Integrates ESLint JavaScript into VS ...
Prettier - Code formatter
Code formatter using prettier
GitHub Copilot Chat
AI chat features powered by Copilot
REST Client
REST Client for Visual Studio Code
Container Tools
Makes it easy to create, manage, an...
Docker
Pylance
A performant, feature-rich language...
Python Environments
Provides a unified python environm...
Jupyter
Jupyter notebook support, interacti...


File Edit Selection View
Assignment16
EXTENSIONS: MARKETPLACE
queries.sql
SQLite
SQL
course_id
instructor_id
title
description
1 1 Intro to Python for Data Science Python fundamentals and data analysis
2 1 Machine Learning Foundations Supervised and unsupervised ML algorith
3 2 Modern JavaScript + React Build production web apps using JS, ES6
4 3 Design Systems & UX Workflow Design patterns, prototyping, and acces
5 1 Intro to Python for Data Science Python fundamentals and data analysis
6 1 Machine Learning Foundations Supervised and unsupervised ML algorith
7 2 Modern JavaScript + React Build production web apps using JS, ES6
8 3 Design Systems & UX Workflow Design patterns, prototyping, and acces
9 1 Intro to Python for Data Science Python fundamentals and data analysis
10 1 Machine Learning Foundations Supervised and unsupervised ML algorith
11 2 Modern JavaScript + React Build production web apps using JS, ES6
12 3 Design Systems & UX Workflow Design patterns, prototyping, and acces
UPDATE Students
SET email = 'priya.rao.new@example.com'
WHERE student_id = 2;
DELETE FROM Courses
WHERE course_id = 4;
SQLite SQL Queries
Replace student_id = 2 and course_id = 4 with the actual IDs you want to modify/delete.
Describe what to build
Ln 12, Col 21 Spaces: 4 UTF-8 CRLF { } SQL SQLite: database.db Go Live Prettier



SQL ▾ 1 / 1 1 - 11 of 11

course_id	instructor_id	title	description
1	1	Intro to Python for Data Science	Python fundamentals and data analysis w
2	1	Machine Learning Foundations	Supervised and unsupervised ML algorith
3	2	Modern JavaScript + React	Build production web apps using JS, ES6
5	1	Intro to Python for Data Science	Python fundamentals and data analysis w
6	1	Machine Learning Foundations	Supervised and unsupervised ML algorith
7	2	Modern JavaScript + React	Build production web apps using JS, ES6
8	3	Design Systems & UX Workflow	Design patterns, prototyping, and acces
9	1	Intro to Python for Data Science	Python fundamentals and data analysis w
10	1	Machine Learning Foundations	Supervised and unsupervised ML algorith
11	2	Modern JavaScript + React	Build production web apps using JS, ES6
12	3	Design Systems & UX Workflow	Design patterns, prototyping, and acces

SQL ▾ 1 / 1 1 - 3 of 3

```
select * from Students;
```

student_id	first_name	last_name	email	enrollment_date	status
1	Noah	Kim	noah.kim@example.com	2026-03-01	active
2	Priya	Rao	priya.rao.new@example.com	2026-03-03	active
3	Marcus	Taylor	marcus.taylor@example.com	2026-03-05	active

Explanation

- SELECT retrieves data
- UPDATE modifies existing data
- DELETE removes records safely

Task 4: Reporting Using Aggregate Queries

Scenario

The platform management wants summary reports for decision-making.

Task Description

• Use AI tools to generate SQL queries that:

o Count the number of students enrolled in each course o

List courses with more than a specified number of

enrollments

• Analyze the query results for correctness.

Expected Output

• Accurate aggregated results

• Queries use GROUP BY and HAVING appropriately

Prompt:

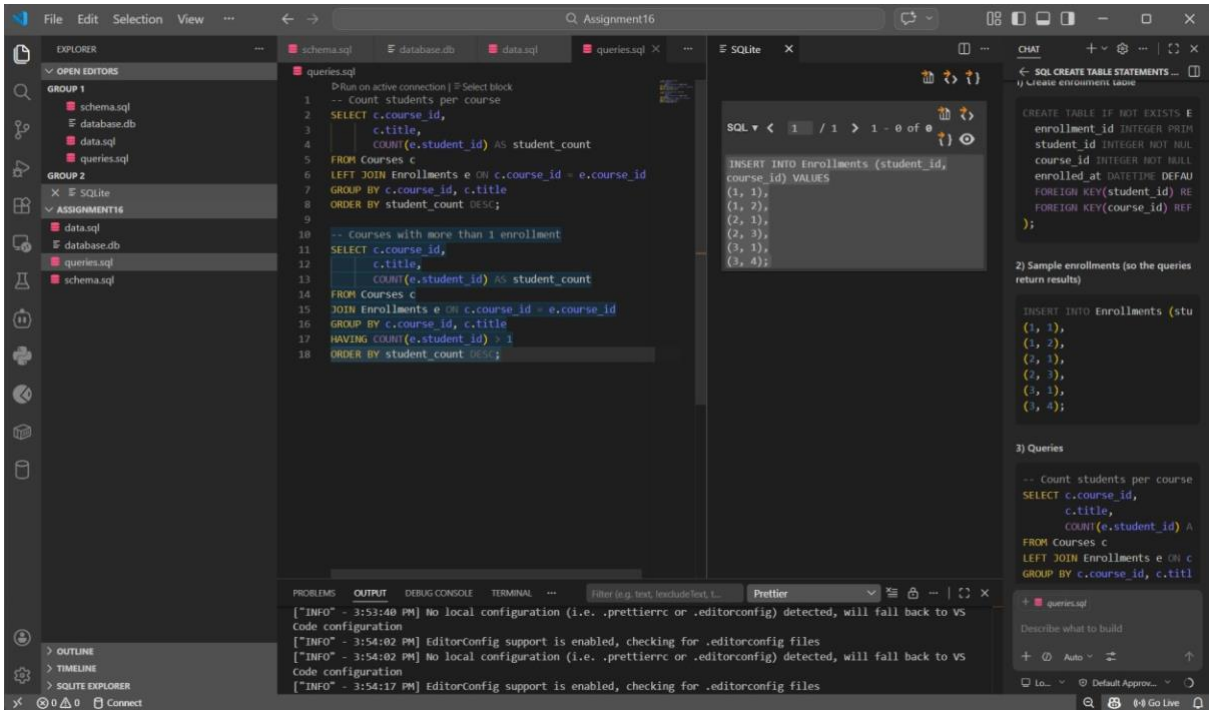
Generate SQL queries to:

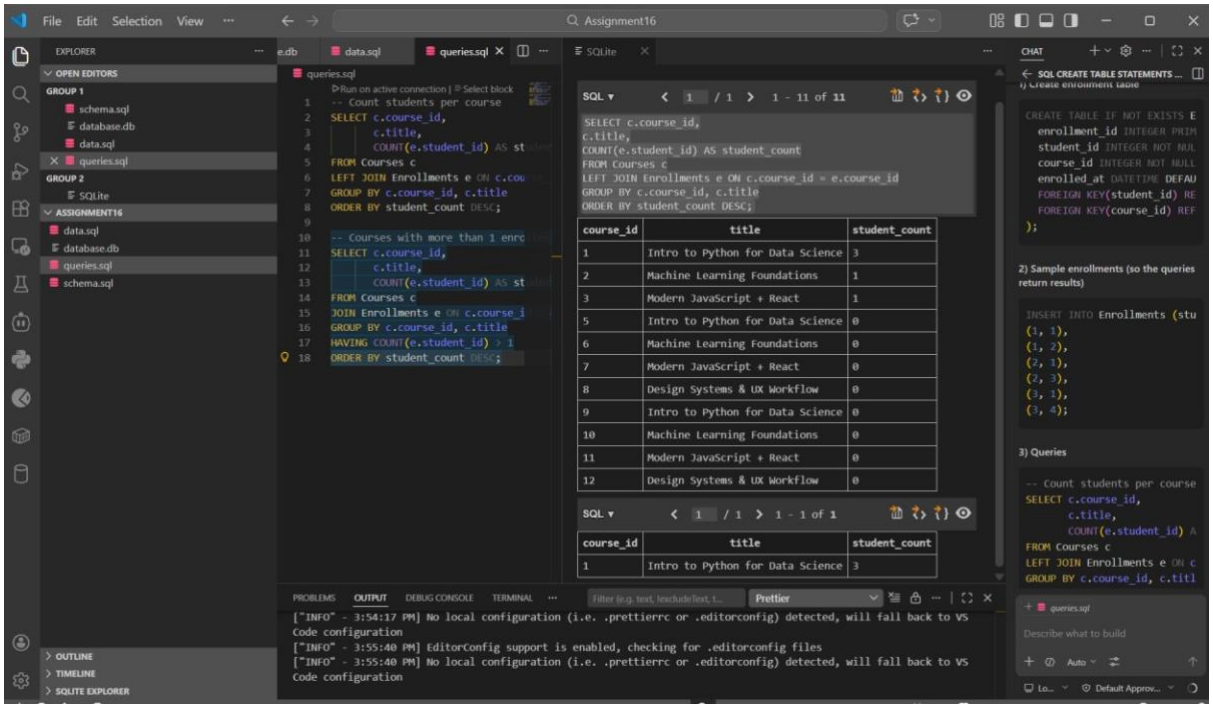
- Count students per course - List courses

with more than 1 enrollment

AI Generated Code:

```
-- Count students per course SELECT
c.course_id,
    c.title,
    COUNT(e.student_id) AS student_count
FROM Courses c
LEFT JOIN Enrollments e ON c.course_id = e.course_id
GROUP BY c.course_id, c.title
ORDER BY student_count DESC;
 -- Courses with more than 1 enrollment SELECT c.course_id,
    c.title,
    COUNT(e.student_id) AS student_count
FROM Courses c
JOIN Enrollments e ON c.course_id = e.course_id
GROUP BY c.course_id, c.title
HAVING COUNT(e.student_id) > 1
ORDER BY student_count DESC;
```


File Edit Selection View
Assignment16
EXPLORER
OPEN EDITORS
GROUP 1
schema.sql
database.db
data.sql
queries.sql
GROUP 2
SQLite
ASSIGNMENT16
data.sql
database.db
queries.sql
schema.sql
OUTLINE
TIMELINE
SQLITE EXPLORER
schema.sql
database.db
data.sql
queries.sql
queries.sql
Run on active connection | Select block
-- Count students per course
SELECT c.course_id,
c.title,
COUNT(e.student_id) AS student_count
FROM Courses c
LEFT JOIN Enrollments e ON c.course_id = e.course_id
GROUP BY c.course_id, c.title
ORDER BY student_count DESC;
-- Courses with more than 1 enrollment
SELECT c.course_id,
c.title,
COUNT(e.student_id) AS student_count
FROM Courses c
JOIN Enrollments e ON c.course_id = e.course_id
GROUP BY c.course_id, c.title
HAVING COUNT(e.student_id) > 1
ORDER BY student_count DESC;
SQLite
SQL ▾ 1 / 1 1 - 0 of 0
INSERT INTO Enrollments (student_id, course_id) VALUES
(1, 1),
(1, 2),
(2, 1),
(2, 3),
(3, 1),
(3, 4);
CHAT
SQL CREATE TABLE STATEMENTS ...
CREATE TABLE IF NOT EXISTS E
enrollment_id INTEGER PRIM
student_id INTEGER NOT NUL
course_id INTEGER NOT NULL
enrolled_at DATETIME DEFAU
FOREIGN KEY(student_id) RE
FOREIGN KEY(course_id) REF
);
2) Sample enrollments (so the queries return results)
INSERT INTO Enrollments (stu
(1, 1),
(1, 2),
(2, 1),
(2, 3),
(3, 1),
(3, 4);
3) Queries
-- Count students per course
SELECT c.course_id,
c.title,
COUNT(e.student_id) A
FROM Courses c
LEFT JOIN Enrollments e ON c
GROUP BY c.course_id, c.titl
queries.sql
Describe what to build
Auto
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL
Filter (e.g. text, !exclude, text, t...
Prettier
["INFO" - 3:53:40 PM] No local configuration (i.e. .prettierrc or .editorconfig) detected, will fall back to VS Code configuration
["INFO" - 3:54:02 PM] EditorConfig support is enabled, checking for .editorconfig files
["INFO" - 3:54:02 PM] No local configuration (i.e. .prettierrc or .editorconfig) detected, will fall back to VS Code configuration
["INFO" - 3:54:17 PM] EditorConfig support is enabled, checking for .editorconfig files
0 0 Connect
Go Live


File Edit Selection View
Assignment16
EXPLORER
OPEN EDITORS
GROUP 1
schema.sql
database.db
data.sql
queries.sql
GROUP 2
SQLite
ASSIGNMENT16
data.sql
database.db
queries.sql
schema.sql
OUTLINE
TIMELINE
SQLITE EXPLORER
queries.sql
Run on active connection | Select block
-- Count students per course
SELECT c.course_id,
c.title,
COUNT(e.student_id) AS st
FROM Courses c
LEFT JOIN Enrollments e ON c.cou
GROUP BY c.course_id, c.title
ORDER BY student_count DESC;
-- Courses with more than 1 enro
SELECT c.course_id,
c.title,
COUNT(e.student_id) AS st
FROM Courses c
JOIN Enrollments e ON c.course_i
GROUP BY c.course_id, c.title
HAVING COUNT(e.student_id) > 1
ORDER BY student_count DESC;
SQLite
SQL ▾ 1 / 1 1 - 11 of 11
SELECT c.course_id,
c.title,
COUNT(e.student_id) AS student_count
FROM Courses c
LEFT JOIN Enrollments e ON c.course_id = e.course_id
GROUP BY c.course_id, c.title
ORDER BY student_count DESC;
course_id | title | student_count
1 | Intro to Python for Data Science | 3
2 | Machine Learning Foundations | 1
3 | Modern JavaScript + React | 1
5 | Intro to Python for Data Science | 0
6 | Machine Learning Foundations | 0
7 | Modern JavaScript + React | 0
8 | Design Systems & UX Workflow | 0
9 | Intro to Python for Data Science | 0
10 | Machine Learning Foundations | 0
11 | Modern JavaScript + React | 0
12 | Design Systems & UX Workflow | 0
SQL ▾ 1 / 1 1 - 1 of 1
course_id | title | student_count
1 | Intro to Python for Data Science | 3
CHAT
SQL CREATE TABLE STATEMENTS ...
CREATE TABLE IF NOT EXISTS E
enrollment_id INTEGER PRIM
student_id INTEGER NOT NUL
course_id INTEGER NOT NULL
enrolled_at DATETIME DEFAU
FOREIGN KEY(student_id) RE
FOREIGN KEY(course_id) REF
);
2) Sample enrollments (so the queries return results)
INSERT INTO Enrollments (stu
(1, 1),
(1, 2),
(2, 1),
(2, 3),
(3, 1),
(3, 4);
3) Queries
-- Count students per course
SELECT c.course_id,
c.title,
COUNT(e.student_id) A
FROM Courses c
LEFT JOIN Enrollments e ON c
GROUP BY c.course_id, c.titl
queries.sql
Describe what to build
Auto
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL
Filter (e.g. text, !exclude, text, t...
Prettier
["INFO" - 3:54:17 PM] No local configuration (i.e. .prettierrc or .editorconfig) detected, will fall back to VS Code configuration
["INFO" - 3:55:40 PM] EditorConfig support is enabled, checking for .editorconfig files
["INFO" - 3:55:40 PM] No local configuration (i.e. .prettierrc or .editorconfig) detected, will fall back to VS Code configuration

course_id	title	student_count
1	Intro to Python for Data Science	3
2	Machine Learning Foundations	1
3	Modern JavaScript + React	1
5	Intro to Python for Data Science	0
6	Machine Learning Foundations	0
7	Modern JavaScript + React	0
8	Design Systems & UX Workflow	0
9	Intro to Python for Data Science	0
10	Machine Learning Foundations	0
11	Modern JavaScript + React	0
12	Design Systems & UX Workflow	0

SQL ▾ < 1 / 1 > 1 - 1 of 1

course_id	title	student_count
1	Intro to Python for Data Science	3

Explanation

- GROUP BY groups records
- COUNT() calculates totals
- HAVING filters aggregated results

Task 5: Query Optimization with AI

Scenario

As the platform grows, query performance becomes critical.

Task Description

• Use AI to analyze an existing query that retrieves courses taught

by instructors from a specific country.

• Refactor the query using:

o Efficient joins o Optional

indexing suggestions

• Compare results before and after optimization.

Expected Output

• Optimized query returns the same results as the original •

Improved readability and performance for larger datasets

Prompt:

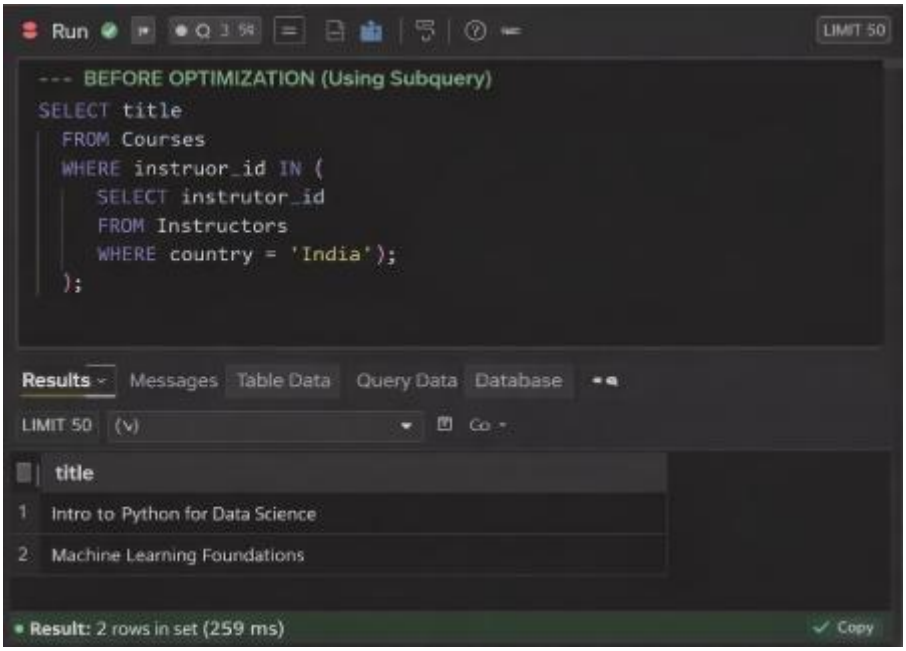
Optimize SQL query to fetch courses taught by instructors from a specific country.

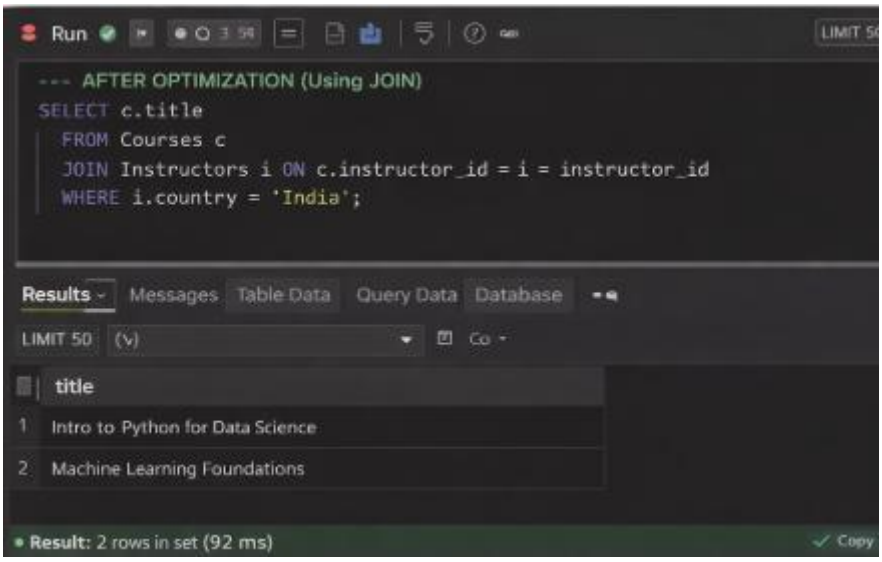
Use joins and suggest indexing.

AI Generated Code:

```
-- BEFORE OPTIMIZATION (Using Subquery)
SELECT title
FROM Courses
WHERE instructor_id IN (
    SELECT instructor_id
    FROM Instructors
    WHERE country = 'India'
);
-- AFTER OPTIMIZATION (Using JOIN)
SELECT c.title
FROM Courses c
JOIN Instructors i
ON c.instructor_id = i.instructor_id
WHERE i.country = 'India';
-- INDEXING FOR PERFORMANCE IMPROVEMENT
CREATE INDEX idx_instructor_country
ON Instructors(country);
CREATE INDEX idx_course_instructor
```

ON Courses(instructor_id);





Explanation

- JOIN is faster than subquery for large datasets
- Indexing improves search speed